VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Recruitment Management System

A9512 – Database Management System Laboratory

Submitted by

A.Aditya Kumari – 26885A0501

G.Rani – 26885A0505

K.Sai Nikhitha – 26885A0506

Course Facilitator

Ms. Shaik Shahanaz
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to JNTUH,
Approved by AICTE, Accredited by NAAC with A++ Grade
Kacharam, Shamshabad, Hyderabad-501 218,

Telangana, India.

Vibrant Vardhaman — 25 Years of

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Recruitment Management System"** is carried out by **"A.Aditya Kumari (26885A0501),G.Rani (26885A0505),K.Sai Nikhitha (26885A0506)"** towards **A9512 – Database Management System Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology in Computer Science and Engineering** during the Academic year 2026–27.

Course Facilitator	Academic Head	Head of the Department
Ms. Shaik Shahanaz	**Dr. R. Karthikeyan**	**Prof. V. Vijayalatha**
Assistant Professor, CSE	Professor, CSE	HoD, CSE

Contents

1 Introduction **5**

2 Problem Analysis **5**

2.1 Problem Statement 5

2.2 Users and Assumptions 5

2.3 Functional Requirements 6

3 ER Model **6**

3.1 Entities and Attributes 6

3.2 Relationships 7

3.3 Cardinality and Participation 7

3.4 ER Diagram 8

4 Relational Model **8**

4.1 Relational Schema 8

4.2 Primary Keys and Foreign Keys 9

5 Normalization **9**

5.1 Functional Dependencies 9

5.2 First Normal Form (1NF) 10

5.3 Second Normal Form (2NF) 10

5.4 Third Normal Form (3NF) 10

6 SQL DDL Commands **10**

6.1 Table Creation 11

6.2 Constraints 13

6.3 ALTER Operation 13

6.4 DROP Operation 13

7 SQL DML Commands **13**

7.1 INSERT 13

7.2 UPDATE 14

7.3 DELETE 14

8 SQL Queries **14**

8.1 Basic SELECT Queries 14

8.2 Joins 14

8.3 Aggregate Functions 15

8.4 GROUP BY and HAVING 15

8.5 Subqueries 15
8.6 Views 15

9 Transactions and Concurrency **16**
9.1 Transaction Management 16
9.2 COMMIT, ROLLBACK and SAVEPOINT 16
9.3 Concurrency Control 16

10 PL/SQL Implementation **16**
10.1 Functions 17
10.2 Stored Procedures 17
10.3 Cursors 17
10.4 Triggers 18

11 File Organization and Indexing **18**
11.1 File Organization 18
11.2 Indexing 19

12 Testing **19**

13 Recruitment Reports **19**
13.1 Open Vacancies Report 19
13.2 Application Status Report 20
13.3 Interview Schedule Report 20
13.4 Selected Candidates and Offers Report 20

14 Results and Discussion **20**

15 Conclusion **21**

16 SDG Mapping **21**

17 Future Scope **22**

Bibliography **22**

Abstract

The Recruitment Management System is a database management project designed to organize and streamline recruitment activities within an organization. It maintains information about job vacancies, departments, recruiters, applicants, applications, interview schedules, interview feedback, and job offers. A centralized relational database replaces scattered spreadsheets and paper records, reduces data duplication, and improves the accuracy and speed of recruitment operations.

The system enables recruiters to publish job openings, review applicant details, track application status, schedule interviews, record feedback, and manage offers. Applicants can be associated with multiple job applications, while each application follows a defined recruitment process. Primary keys, foreign keys, constraints, and normalization are used to maintain consistency and integrity. Reports can summarize vacancies, applications, shortlisted candidates, interview outcomes, and selected candidates.

Keywords: Recruitment Management System, DBMS, SQL, applicants, vacancies, applications, interviews, database normalization.

1 Introduction

Recruitment is an important organizational process that identifies, attracts, evaluates, and selects suitable candidates for available positions. Human resource teams handle job descriptions, applicant profiles, resumes, application status, interview schedules, interviewer feedback, and offer details. When this information is maintained manually or in separate files, it can lead to duplicate records, missed interviews, delayed communication, and difficulty in tracking candidates.

The Recruitment Management System provides a centralized database-based approach to manage these activities in an organized manner. It stores department and job details, maintains applicant records, tracks applications through different stages, and records interview results and job offers. Database concepts such as ER modeling, relational schema design, normalization, constraints, SQL queries, transactions, PL/SQL procedures, triggers, and indexing help create a reliable and maintainable system.

The proposed system supports recruiters and human resource managers by keeping recruitment information connected and easy to retrieve. It also helps generate reports for open positions, application progress, interview outcomes, and hiring results. The database design used in this report contains the entities Department, Recruiter, Job, Applicant, Application, Interview, Interviewer, and Job_Offer. These entities are connected through primary and foreign keys to reduce inconsistent records and support accurate reporting.

2 Problem Analysis

Organizations often manage recruitment details through emails, spreadsheets, and paper forms. This makes it difficult to maintain a single record for each applicant, determine the latest application status, coordinate interview schedules, and prepare accurate hiring reports. Repeated data entry may also create duplicate applicant profiles or inconsistent job information.

2.1 Problem Statement

A recruitment team needs to manage job vacancies, applicant details, applications, interviews, feedback, and offers. When these records are stored in separate files, tracking a candidate from application to selection becomes difficult. The proposed Recruitment Management System uses a centralized relational database to store and connect recruitment information, improve retrieval, reduce duplication, and support consistent decision-making.

2.2 Users and Assumptions

The main users are the administrator, recruiter, interviewer, and human resource manager. The administrator maintains user and organizational information. Recruiters create vacancies and review applications. Interviewers record interview feedback, while the HR manager monitors recruitment progress and hiring reports.

The system uses the following assumptions:

- Each department has a unique department identifier.
- Each job vacancy belongs to one department and is managed by a recruiter.
- An applicant can submit applications for multiple vacancies.
- Each application links one applicant to one job vacancy.
- An application may have multiple interview records when required.
- A job offer is recorded only for a selected application.

2.3 Functional Requirements

The system supports creating and updating department, recruiter, job, applicant, application, interview, and offer records. Recruiters can publish vacancies, search applicants, update application stages, and schedule interviews. Interviewers can record feedback and recommendations. HR managers can view vacancy status, application counts, interview outcomes, and selected candidates. The database uses constraints to prevent invalid references and supports reports for recruitment planning.

3 ER Model

The Entity-Relationship (ER) model represents the data required by the Recruitment Management System and the relationships among its entities. The principal entities are Department, Recruiter, Job, Applicant, Application, Interview, Interviewer, and Job_Offer. Each entity has attributes that describe its information and a primary key that uniquely identifies each record.

3.1 Entities and Attributes

Department stores organizational departments. Recruiter stores staff responsible for recruitment. Job stores vacancy details such as title, description, number of openings, and status. Applicant stores candidate contact and qualification details. Application links an applicant with a job and tracks the application date and status. Interview records interview dates and results. Interviewer stores the people conducting interviews, and Job_Offer stores the offer made to a selected candidate.

Table 3.1: Entities and Attributes of the System

Entity	**Attributes**	**Keys**
Department	Department_ID, Department_Name, Location	PK: Department_ID
Recruiter	Recruiter_ID, Name, Email, Phone, Department_ID	PK: Recruiter_ID; FK: Department_ID

Entity	Attributes	Keys
Job	Job_ID, Title, Description, Posting_Date, Closing_Date, Openings, Status, Department_ID, Recruiter_ID	PK: Job_ID; FKs: Department_ID, Recruiter_ID
Applicant	Applicant_ID, First_Name, Last_Name, Email, Phone, Qualification, Experience_Years, Resume_Path	PK: Applicant_ID
Application	Application_ID, Applicant_ID, Job_ID, Application_Date, Status, Score	PK: Application_ID; FKs: Applicant_ID, Job_ID
Interviewer	Interviewer_ID, Name, Email, Department_ID	PK: Interviewer_ID; FK: Department_ID
Interview	Interview_ID, Application_ID, Interviewer_ID, Interview_Date, Mode, Feedback, Result	PK: Interview_ID; FKs: Application_ID, Interviewer_ID
Job_Offer	Offer_ID, Application_ID, Offer_Date, Salary, Joining_Date, Offer_Status	PK: Offer_ID; FK: Application_ID

3.2 Relationships

A department can have many recruiters, interviewers, and jobs. Each job belongs to one department and is managed by one recruiter. An applicant can submit many applications, and each application refers to one applicant and one job. An application can have multiple interviews, and an interviewer can conduct many interviews. A selected application may receive a job offer; the offer is linked to the relevant application.

3.3 Cardinality and Participation

Table 3.2: Relationships, Cardinality and Participation

Relationship	Entities	Cardinality	Participation
Belongs to	Department–Recruiter	1:N	Recruiter: total
Publishes	Department–Job	1:N	Job: total
Manages	Recruiter–Job	1:N	Job: total
Submits	Applicant–Application	1:N	Application: total

Relationship	Entities	Cardinality	Participation
Receives	Job–Application	1:N	Application: total
Schedules	Application–Interview	1:N	Interview: total
Conducts	Interviewer–Interview	1:N	Interview: total
Results in	Application–Job_Offer	1:0..1	Offer: linked to application

3.4 ER Diagram

The ER diagram should show Department connected to Recruiter, Interviewer, and Job; Recruiter connected to Job; Applicant and Job connected through Application; Application connected to Interview and Job_Offer; and Interviewer connected to Interview. These relationships form the basis for converting the ER model into relational tables. If an ER image is available, insert it here using the following editable placeholder:

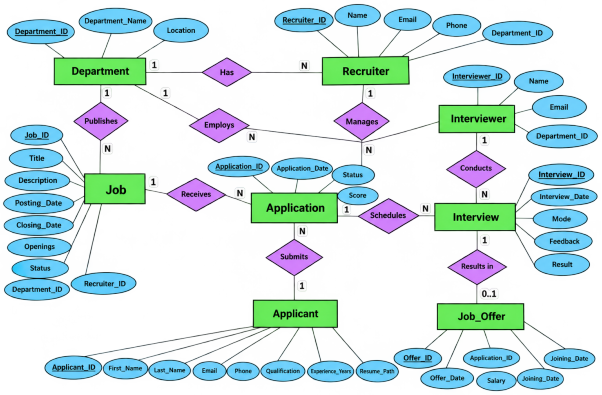


Figure 3.1: ER Diagram of Recruitment Management System

4 Relational Model

The relational model stores recruitment information in tables. Primary keys uniquely identify records, while foreign keys connect related tables and maintain referential integrity.

4.1 Relational Schema

The main relational schema is listed below. PK indicates a primary key and FK indicates a foreign key.

- Department(Department_ID PK, Department_Name, Location)
- Recruiter(Recruiter_ID PK, Name, Email, Phone, Department_ID FK)
- Job(Job_ID PK, Title, Description, Posting_Date, Closing_Date, Openings, Status, Department_ID FK, Recruiter_ID FK)

- Applicant(Applicant_ID PK, First_Name, Last_Name, Email, Phone, Qualification, Experience_Years, Resume_Path)
- Application(Application_ID PK, Applicant_ID FK, Job_ID FK, Application_Date, Status, Score)
- Interviewer(Interviewer_ID PK, Name, Email, Department_ID FK)
- Interview(Interview_ID PK, Application_ID FK, Interviewer_ID FK, Interview_Date, Mode, Feedback, Result)
- Job_Offer(Offer_ID PK, Application_ID FK, Offer_Date, Salary, Joining_Date, Offer_Status)

4.2 Primary Keys and Foreign Keys

Each table has a primary key. Foreign keys link recruiters and jobs to departments, applications to applicants and jobs, interviews to applications and interviewers, and offers to applications. Applicant email and recruiter email are declared UNIQUE to avoid duplicate contact records. A unique constraint on Job_Offer.Application_ID can be used when the business rule permits only one offer per application.

Table 4.1: Foreign Keys in the Schema

Table	**Foreign Key**	**References**
Recruiter	Department_ID	Department(Department_ID)
Job	Department_ID	Department(Department_ID)
Job	Recruiter_ID	Recruiter(Recruiter_ID)
Interviewer	Department_ID	Department(Department_ID)
Application	Applicant_ID	Applicant(Applicant_ID)
Application	Job_ID	Job(Job_ID)
Interview	Application_ID	Application(Application_ID)
Interview	Interviewer_ID	Interviewer(Interviewer_ID)
Job_Offer	Application_ID	Application(Application_ID)

5 Normalization

Normalization organizes tables to reduce redundant data and avoid insertion, update, and deletion anomalies. The Recruitment Management System is designed toward Third Normal Form (3NF). Department details are stored separately from jobs and recruiters, applicant details are stored separately from applications, and interview and offer records are maintained in their own tables.

5.1 Functional Dependencies

The principal functional dependencies are:

- Department_ID $\rightarrow$ Department_Name, Location

- Recruiter_ID → Name, Email, Phone, Department_ID
- Job_ID → Title, Description, Posting_Date, Closing_Date, Openings, Status, Department_ID, Recruiter_ID
- Applicant_ID → First_Name, Last_Name, Email, Phone, Qualification, Experience_Years, Resume_Path
- Application_ID → Applicant_ID, Job_ID, Application_Date, Status, Score
- Interview_ID → Application_ID, Interviewer_ID, Interview_Date, Mode, Feedback, Result
- Offer_ID → Application_ID, Offer_Date, Salary, Joining_Date, Offer_Status

These dependencies ensure that non-key attributes depend on the key of their own relation rather than being repeatedly stored in unrelated tables.

5.2 First Normal Form (1NF)

A relation is in First Normal Form when every attribute contains an atomic value and there are no repeating groups. A poorly designed recruitment sheet might store several interview dates or multiple skills in one cell. These values should be separated into individual records or related tables. For example, each interview is stored as a separate row in the Interview table, so each cell contains one value.

5.3 Second Normal Form (2NF)

A relation is in Second Normal Form when it is in 1NF and every non-key attribute depends on the complete primary key. The proposed tables primarily use single-column primary keys, so partial dependencies are avoided. Applicant information depends on Applicant_ID, while job information depends on Job_ID; neither is duplicated as descriptive data in every Application row.

5.4 Third Normal Form (3NF)

A relation is in Third Normal Form when it is in 2NF and non-key attributes do not depend transitively on the primary key. Department names are stored in Department rather than repeated in Job and Recruiter. Applicant details are stored in Applicant, while Application stores only the applicant and job identifiers plus application-specific attributes. This separation reduces update anomalies and improves maintainability.

6 SQL DDL Commands

Data Definition Language (DDL) commands create and modify database structures. The examples below use Oracle SQL syntax and define tables with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, and DEFAULT constraints.

6.1 Table Creation

```
CREATE TABLE Department (
  Department_ID NUMBER PRIMARY KEY,
  Department_Name VARCHAR2(100) NOT NULL UNIQUE,
  Location VARCHAR2(100)
);

CREATE TABLE Recruiter (
  Recruiter_ID NUMBER PRIMARY KEY,
  Name VARCHAR2(100) NOT NULL,
  Email VARCHAR2(100) UNIQUE NOT NULL,
  Phone VARCHAR2(15),
  Department_ID NUMBER NOT NULL,
  FOREIGN KEY (Department_ID) REFERENCES Department(Department_ID)
);

CREATE TABLE Job (
  Job_ID NUMBER PRIMARY KEY,
  Title VARCHAR2(100) NOT NULL,
  Description VARCHAR2(500),
  Posting_Date DATE DEFAULT SYSDATE,
  Closing_Date DATE,
  Openings NUMBER DEFAULT 1 CHECK (Openings > 0),
  Status VARCHAR2(20) DEFAULT 'Open'
    CHECK (Status IN ('Open','Closed','On␣Hold')),
  Department_ID NUMBER NOT NULL,
  Recruiter_ID NUMBER NOT NULL,
  CHECK (Closing_Date IS NULL OR Closing_Date >= Posting_Date),
  FOREIGN KEY (Department_ID) REFERENCES Department(Department_ID),
  FOREIGN KEY (Recruiter_ID) REFERENCES Recruiter(Recruiter_ID)
);

CREATE TABLE Applicant (
  Applicant_ID NUMBER PRIMARY KEY,
  First_Name VARCHAR2(50) NOT NULL,
  Last_Name VARCHAR2(50),
  Email VARCHAR2(100) UNIQUE NOT NULL,
  Phone VARCHAR2(15),
  Qualification VARCHAR2(100),
  Experience_Years NUMBER(4,1) DEFAULT 0 CHECK (Experience_Years >= 0),
  Resume_Path VARCHAR2(255)
);

CREATE TABLE Application (
  Application_ID NUMBER PRIMARY KEY,
```

```
  Applicant_ID NUMBER NOT NULL,
  Job_ID NUMBER NOT NULL,
  Application_Date DATE DEFAULT SYSDATE,
  Status VARCHAR2(20) DEFAULT 'Applied'
    CHECK (Status IN ('Applied','Shortlisted','Rejected','Selected','Withdrawn'
        )),
  Score NUMBER(5,2) CHECK (Score BETWEEN 0 AND 100),
  UNIQUE (Applicant_ID, Job_ID),
  FOREIGN KEY (Applicant_ID) REFERENCES Applicant(Applicant_ID),
  FOREIGN KEY (Job_ID) REFERENCES Job(Job_ID)
);

CREATE TABLE Interviewer (
  Interviewer_ID NUMBER PRIMARY KEY,
  Name VARCHAR2(100) NOT NULL,
  Email VARCHAR2(100) UNIQUE,
  Department_ID NUMBER,
  FOREIGN KEY (Department_ID) REFERENCES Department(Department_ID)
);

CREATE TABLE Interview (
  Interview_ID NUMBER PRIMARY KEY,
  Application_ID NUMBER NOT NULL,
  Interviewer_ID NUMBER NOT NULL,
  Interview_Date DATE NOT NULL,
  Mode VARCHAR2(20) CHECK (Mode IN ('Online','In␣Person','Telephone')),
  Feedback VARCHAR2(500),
  Result VARCHAR2(20) CHECK (Result IN ('Pending','Pass','Fail')),
  FOREIGN KEY (Application_ID) REFERENCES Application(Application_ID),
  FOREIGN KEY (Interviewer_ID) REFERENCES Interviewer(Interviewer_ID)
);

CREATE TABLE Job_Offer (
  Offer_ID NUMBER PRIMARY KEY,
  Application_ID NUMBER NOT NULL UNIQUE,
  Offer_Date DATE DEFAULT SYSDATE,
  Salary NUMBER(12,2) CHECK (Salary > 0),
  Joining_Date DATE,
  Offer_Status VARCHAR2(20) DEFAULT 'Pending'
    CHECK (Offer_Status IN ('Pending','Accepted','Declined','Expired')),
  FOREIGN KEY (Application_ID) REFERENCES Application(Application_ID)
);
```

6.2 Constraints

Primary keys uniquely identify rows and foreign keys connect related tables. NOT NULL ensures required data is provided. UNIQUE prevents duplicate email addresses and duplicate applications by the same applicant for the same job. CHECK restricts fields to valid values, such as non-negative experience, valid application statuses, and positive offered salary. DEFAULT supplies initial values such as the posting date and application status.

6.3 ALTER Operation

ALTER modifies the structure of an existing table.

```
ALTER TABLE Applicant ADD Skills VARCHAR2(300);
ALTER TABLE Recruiter MODIFY Phone VARCHAR2(20);
```

6.4 DROP Operation

DROP removes a table and its stored records. It should be used carefully when dependent data exists.

```
DROP TABLE Interview CASCADE CONSTRAINTS;
```

7 SQL DML Commands

Data Manipulation Language (DML) commands insert, modify, and delete records. Sample statements below use fictional recruitment data.

7.1 INSERT

```
INSERT INTO Department VALUES (1, 'Computer␣Science', 'Hyderabad');
INSERT INTO Department VALUES (2, 'Human␣Resources', 'Hyderabad');
INSERT INTO Recruiter VALUES (101, 'Anita␣Rao', 'anita@example.com', '
   9000011111', 2);
INSERT INTO Recruiter VALUES (102, 'Rahul␣Kumar', 'rahul@example.com', '
   9000022222', 1);
INSERT INTO Job VALUES (201, 'Software␣Developer', 'Develop␣and␣maintain␣
   applications',
  DATE '2026-10-01', DATE '2026-11-15', 3, 'Open', 1, 102);
INSERT INTO Job VALUES (202, 'HR␣Executive', 'Coordinate␣recruitment␣
   activities',
  DATE '2026-10-02', DATE '2026-11-20', 2, 'Open', 2, 101);
INSERT INTO Applicant VALUES (301, 'Ravi', 'Kumar', 'ravi@example.com', '
   9876543210',
  'B.Tech␣CSE', 1.0, 'resumes/ravi.pdf');
INSERT INTO Applicant VALUES (302, 'Meena', 'Rao', 'meena@example.com', '
   9123456780',
  'MBA␣HR', 2.0, 'resumes/meena.pdf');
```

```
INSERT INTO Interviewer VALUES (401, 'Sandeep␣Reddy', 'sandeep@example.com',
    1);
INSERT INTO Application VALUES (501, 301, 201, DATE '2026-10-04', 'Shortlisted
    ', 82);
INSERT INTO Application VALUES (502, 302, 202, DATE '2026-10-05', 'Applied',
    75);
INSERT INTO Interview VALUES (601, 501, 401, DATE '2026-10-15', 'Online',
  'Strong␣technical␣fundamentals', 'Pass');
INSERT INTO Job_Offer VALUES (701, 501, DATE '2026-10-18', 650000,
  DATE '2026-11-10', 'Pending');
COMMIT;
```

7.2 UPDATE

UPDATE modifies existing records. For example, a recruiter may shortlist an applicant or close a vacancy after filling the position.

```
UPDATE Application SET Status = 'Shortlisted' WHERE Application_ID = 502;
UPDATE Job SET Status = 'Closed' WHERE Job_ID = 202;
UPDATE Job_Offer SET Offer_Status = 'Accepted' WHERE Offer_ID = 701;
```

7.3 DELETE

DELETE removes records. Referenced records may be protected by foreign-key constraints, preserving related recruitment history.

```
DELETE FROM Interview WHERE Interview_ID = 601;
DELETE FROM Job WHERE Job_ID = 202;
```

8 SQL Queries

SQL queries retrieve and analyze information stored in the Recruitment Management System. SELECT statements, joins, aggregate functions, grouping, subqueries, and views can support recruitment decisions.

8.1 Basic SELECT Queries

```
SELECT * FROM Job;
SELECT First_Name, Last_Name, Qualification FROM Applicant;
SELECT * FROM Job WHERE Status = 'Open';
SELECT * FROM Application WHERE Status = 'Shortlisted';
```

8.2 Joins

Joins retrieve related information from multiple tables.

```
-- Applications with applicant and job details
SELECT A.Application_ID, P.First_Name, P.Last_Name, J.Title, A.Status
FROM Application A
```

```
JOIN Applicant P ON A.Applicant_ID = P.Applicant_ID
JOIN Job J ON A.Job_ID = J.Job_ID;

-- Interview schedules with interviewer details
SELECT I.Interview_Date, P.First_Name, J.Title, V.Name AS Interviewer
FROM Interview I
JOIN Application A ON I.Application_ID = A.Application_ID
JOIN Applicant P ON A.Applicant_ID = P.Applicant_ID
JOIN Job J ON A.Job_ID = J.Job_ID
JOIN Interviewer V ON I.Interviewer_ID = V.Interviewer_ID;
```

8.3 Aggregate Functions

COUNT, AVG, MAX, MIN, and other aggregate functions summarize recruitment data.

```
SELECT COUNT(*) AS Total_Jobs FROM Job;
SELECT COUNT(*) AS Total_Applicants FROM Applicant;
SELECT AVG(Score) AS Average_Score FROM Application;
SELECT MAX(Salary) AS Highest_Offer FROM Job_Offer;
```

8.4 GROUP BY and HAVING

GROUP BY groups records and HAVING filters grouped results.

```
SELECT Status, COUNT(*) AS Total_Applications
FROM Application GROUP BY Status;

SELECT Job_ID, COUNT(*) AS Applicants
FROM Application GROUP BY Job_ID HAVING COUNT(*) > 1;
```

8.5 Subqueries

A subquery is a query nested inside another query. The following examples identify applicants with scores above the average and jobs that have received no applications.

```
SELECT Applicant_ID, Score FROM Application
WHERE Score > (SELECT AVG(Score) FROM Application);

SELECT Job_ID, Title FROM Job J
WHERE NOT EXISTS (
  SELECT 1 FROM Application A WHERE A.Job_ID = J.Job_ID
);
```

8.6 Views

A view is a virtual table created from a query. This view combines application, applicant, and vacancy information.

```
CREATE VIEW Application_Details AS
```

```
SELECT A.Application_ID, P.First_Name, P.Last_Name,
       J.Title, A.Application_Date, A.Status
FROM Application A
JOIN Applicant P ON A.Applicant_ID = P.Applicant_ID
JOIN Job J ON A.Job_ID = J.Job_ID;

SELECT * FROM Application_Details;
```

9 Transactions and Concurrency

Transactions group database operations into a logical unit. In recruitment, a process may update an application status, record an interview result, and create an offer. Transaction management helps ensure that related changes are committed together or rolled back if an error occurs.

9.1 Transaction Management

A transaction begins when database operations are performed and ends when changes are committed or cancelled. For example, changing an application to selected and creating its offer should be handled consistently so that incomplete recruitment records are avoided.

9.2 COMMIT, ROLLBACK and SAVEPOINT

COMMIT permanently saves changes. ROLLBACK cancels uncommitted changes, while SAVEPOINT creates a point to which a transaction can be rolled back.

```
SAVEPOINT application_update;
UPDATE Application SET Status = 'Selected' WHERE Application_ID = 501;
INSERT INTO Job_Offer VALUES (702, 501, SYSDATE, 700000, DATE '2026-11-15', '
    Pending');
-- Use COMMIT after verifying the operation, or ROLLBACK if it fails.
ROLLBACK TO application_update;
COMMIT;
```

9.3 Concurrency Control

Concurrency control manages simultaneous transactions accessing the same data. For example, two recruiters may try to update the same application. Locking relevant records during a critical update can reduce conflicting changes.

```
SELECT * FROM Application WHERE Application_ID = 501 FOR UPDATE;
```

Proper transaction management helps preserve data consistency when several users work with recruitment records at the same time.

10 PL/SQL Implementation

PL/SQL extends SQL with variables, conditions, loops, procedures, functions, cursors, and triggers. In the Recruitment Management System, it can automate application count-

ing, status updates, and recruitment-related checks.

10.1 Functions

A function can return the number of applications submitted by an applicant.

```
CREATE OR REPLACE FUNCTION Get_Application_Count(P_ID NUMBER)
RETURN NUMBER IS
  Total_Applications NUMBER;
BEGIN
  SELECT COUNT(*) INTO Total_Applications
  FROM Application WHERE Applicant_ID = P_ID;
  RETURN Total_Applications;
END;
/

SELECT Get_Application_Count(301) FROM DUAL;
```

10.2 Stored Procedures

A stored procedure performs a specific database operation. This procedure updates an application's status after validating the supplied status value.

```
CREATE OR REPLACE PROCEDURE Update_Application_Status(
  P_Application_ID NUMBER,
  P_Status VARCHAR2
) AS
BEGIN
  IF P_Status NOT IN ('Applied','Shortlisted','Rejected','Selected','Withdrawn
      ') THEN
    RAISE_APPLICATION_ERROR(-20001, 'Invalid␣application␣status');
  END IF;
  UPDATE Application
  SET Status = P_Status
  WHERE Application_ID = P_Application_ID;
  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'Application␣not␣found');
  END IF;
  COMMIT;
END;
/

EXEC Update_Application_Status(502, 'Shortlisted');
```

10.3 Cursors

A cursor processes rows returned by a query one at a time. The following PL/SQL block displays all open job vacancies.

```
DECLARE
  CURSOR Job_Cursor IS
    SELECT Job_ID, Title FROM Job WHERE Status = 'Open';
BEGIN
  FOR J IN Job_Cursor LOOP
    DBMS_OUTPUT.PUT_LINE(J.Job_ID || ' - ' || J.Title);
  END LOOP;
END;
/
```

10.4 Triggers

A trigger executes automatically when a specified database event occurs. The example below assigns the default application status if an inserted record does not provide one. The table also defines a DEFAULT constraint, which is normally sufficient for this rule.

```
CREATE OR REPLACE TRIGGER Set_Application_Status
BEFORE INSERT ON Application
FOR EACH ROW
BEGIN
  IF :NEW.Status IS NULL THEN
    :NEW.Status := 'Applied';
  END IF;
END;
/
```

11 File Organization and Indexing

File organization describes how records are physically arranged in storage so that they can be accessed and updated efficiently. In the Recruitment Management System, records are stored in relational tables such as Applicant, Job, Application, Interview, and Job_Offer. The DBMS manages the physical storage of these tables, while suitable organization and indexes help improve retrieval performance.

11.1 File Organization

The common file organization methods are heap, sequential, hash, and indexed organization. Heap organization stores records without a fixed order and is suitable when records are frequently inserted. Sequential organization stores records in a particular order and can support ordered processing. Hash organization uses a hash function to locate records quickly when searching by a key. Indexed organization maintains a separate index that points to records and supports faster searches on selected fields.

For this project, the DBMS manages the underlying table files. Applicant and application records may be inserted frequently, while recruiters often search jobs by status, applications by status, and interviews by date. Choosing suitable indexes for these search fields can reduce the number of records that must be examined.

11.2 Indexing

An index is a data structure that helps the DBMS find records without scanning the entire table. Indexes are useful for frequently searched columns such as job status, application status, applicant email, and interview date. However, excessive indexing consumes storage and can slow insert, update, and delete operations, so indexes should be created based on actual query requirements.

```
CREATE INDEX idx_job_status ON Job(Status);
CREATE INDEX idx_application_status ON Application(Status);
CREATE INDEX idx_interview_date ON Interview(Interview_Date);
CREATE INDEX idx_applicant_email ON Applicant(Email);
```

12 Testing

Testing checks whether the database operations of the Recruitment Management System work as expected. The following test cases cover valid data entry, constraint validation, application tracking, interview records, and report queries.

Test Case	**Expected Result**	**Purpose**
Insert a department	Department record is saved	Verify valid data insertion
Insert an applicant with a duplicate email	Database rejects the duplicate	Check UNIQUE constraint
Create an application for an existing applicant and job	Application is saved with linked IDs	Check foreign-key relationships
Enter an invalid application status	Database rejects the invalid value	Check CHECK constraint
Update an application status	The new status is stored	Verify update operation
Display open vacancies	Only jobs with Open status are listed	Verify report query
Retrieve interview details	Interview and related applicant/job details are displayed	Verify joins

These tests help verify that records are stored correctly, invalid data is restricted, related tables remain connected, and recruitment information can be retrieved through SQL queries.

13 Recruitment Reports

Reports summarize recruitment progress and help recruiters and HR managers make informed decisions.

13.1 Open Vacancies Report

This report lists vacancies that are currently open.

```
SELECT Job_ID, Title, Posting_Date, Closing_Date, Openings
FROM Job WHERE Status = 'Open';
```

13.2 Application Status Report

The application status report counts applications at each stage of the recruitment process.

```
SELECT Status, COUNT(*) AS Total_Applications
FROM Application GROUP BY Status;
```

13.3 Interview Schedule Report

The interview report lists upcoming interviews with applicant and job details.

```
SELECT I.Interview_ID, I.Interview_Date, P.First_Name, P.Last_Name, J.Title
FROM Interview I
JOIN Application A ON I.Application_ID = A.Application_ID
JOIN Applicant P ON A.Applicant_ID = P.Applicant_ID
JOIN Job J ON A.Job_ID = J.Job_ID
WHERE I.Interview_Date >= SYSDATE
ORDER BY I.Interview_Date;
```

13.4 Selected Candidates and Offers Report

This report lists selected applicants and their offer details.

```
SELECT P.First_Name, P.Last_Name, J.Title, O.Offer_Date,
      O.Salary, O.Offer_Status
FROM Job_Offer O
JOIN Application A ON O.Application_ID = A.Application_ID
JOIN Applicant P ON A.Applicant_ID = P.Applicant_ID
JOIN Job J ON A.Job_ID = J.Job_ID;
```

These reports provide a summary of vacancies, application progress, interviews, and offers, supporting better recruitment planning.

14 Results and Discussion

The Recruitment Management System provides a structured database for managing departments, recruiters, job vacancies, applicants, applications, interviews, and job offers. SQL commands support the creation, modification, deletion, and retrieval of recruitment records.

The relational design separates applicant, vacancy, interview, and offer details to reduce redundancy. Primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, and DEFAULT constraints help maintain data integrity. Transactions and concurrency control support consistent updates, while functions, procedures, cursors, triggers, and indexes demonstrate additional DBMS capabilities.

The queries and reports provide useful views of open jobs, application stages, interview schedules, and selected candidates. The system forms a database foundation for a recruitment workflow and can be extended with a web interface, secure authentication, resume parsing, and notification services.

15 Conclusion

The Recruitment Management System provides an organized database-based solution for managing recruitment activities. It maintains information about departments, recruiters, vacancies, applicants, applications, interviews, and job offers in a centralized relational database, reducing reliance on scattered records.

The project demonstrates key DBMS concepts, including ER modeling, relational schema design, normalization, SQL DDL and DML commands, queries, transactions, concurrency control, PL/SQL functions, procedures, cursors, triggers, and indexing. These concepts improve data consistency, reduce duplication, and support efficient retrieval and reporting.

The system can help recruitment teams track candidates through different stages and prepare reports for vacancies, interviews, and offers. Future enhancements such as applicant portals, resume parsing, email/SMS notifications, calendar integration, and analytics could make the system more useful in real-world recruitment.

16 SDG Mapping

The Recruitment Management System supports digital record management and can contribute to efficient, transparent, and inclusive recruitment practices.

Table 16.1: SDG Mapping of the Project

SDG Goal	**Contribution of the Project**
SDG 8: Decent Work and Economic Growth	Organizes vacancy and applicant information and supports more efficient hiring processes.
SDG 9: Industry, Innovation and Infrastructure	Applies database technology to digitize recruitment records and workflows.
SDG 10: Reduced Inequalities	A well-designed recruitment process can support consistent evaluation criteria and improve access to vacancy information.
SDG 16: Peace, Justice and Strong Institutions	Structured records and status tracking can improve accountability and traceability in recruitment activities.

The project demonstrates how database technology can support organized digital recruitment processes and reduce dependence on paper-based record keeping.

17 Future Scope

The Recruitment Management System can be enhanced with a candidate portal for registration and application tracking, secure role-based access for recruiters and interviewers, resume parsing, and automated email or SMS notifications. Calendar integration could simplify interview scheduling, while online interview links could be stored with interview records.

Further improvements may include candidate skill matching, analytics for time-to-hire, source-of-applicant reports, interview feedback dashboards, audit logs, and data privacy controls. Integration with job portals and human resource information systems could help make the system suitable for larger organizations.

Bibliography

1. Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, *Database System Concepts*, McGraw-Hill Education.

2. Ramez Elmasri and Shamkant B. Navathe, *Fundamentals of Database Systems*, Pearson.

3. Oracle Corporation, *Oracle Database Documentation*.

4. MySQL Documentation, *MySQL Reference Manual*.

5. GeeksforGeeks, "DBMS and SQL Tutorials," online reference.